

Projeto Full Stack (Parte 1)

Sistema de Gestão de Condomínios

Preparação do Ambiente e Modelagem de Dados

Objetivo

Criar o banco de dados **DBCONDOMINIO**, com as tabelas e relacionamentos necessários.

1. Instalar PostgreSQL + pgAdmin (ou Workbench equivalente)

- Baixar e instalar o **PostgreSQL**: [postgresql.org/download](https://www.postgresql.org/download).
- Durante a instalação, configurar usuário `postgres` e senha.
- Instalar o **pgAdmin** (vem junto) ou usar **Workbench** para PostgreSQL.

2. Criar o Banco de Dados

No Workbench/pgAdmin:

- Abrir a conexão com o servidor.
- Criar novo banco chamado **DBCONDOMINIO**.

Crie as tabelas de acordo com as seguintes entidades:

PESSOAS

- PK: ID_PESSOA
- NOME, TIPO_PESSOA (FÍSICA/JURÍDICA), CPF_CNPJ, DATA_CADASTRO

CONTATOS

- PK: ID_CONTATO
- FK: ID_PESSOA → PESSOAS
- TIPO_CONTATO, VALOR_CONTATO

ENDERECOS

- PK: ID_ENDERECO
- FK: ID_PESSOA → PESSOAS
- LOGRADOURO, NUMERO, BAIRRO, CIDADE, UF, CEP

MORADORES

- PK: ID_MORADOR
- FK: ID_PESSOA → PESSOAS
- FK: ID_UNIDADE → UNIDADES

FUNCIONARIOS

- PK: ID_FUNCIONARIO
- FK: ID_PESSOA → PESSOAS
- FUNCAO, DATA_ADMISSAO, SALARIO

FORNECEDORES

- PK: ID_FORNECEDOR
- FK: ID_PESSOA → PESSOAS
- AREA_ATUACAO

VISITANTES

- PK: ID_VISITANTE
- FK: ID_PESSOA → PESSOAS
- DOCUMENTO

UNIDADES

- PK: ID_UNIDADE
- NUM_UNIDADE, BLOCO, TIPO, AREA_TOTAL

AREAS_COMUNS

- PK: ID_AREA_COMUM
- NOME_AREA, DESCR_AREA, CAPACIDADE

RESERVAS

- PK: ID_RESERVA
- FK: ID_AREA_COMUM → AREAS_COMUNS
- FK: ID_MORADOR → MORADORES
- DATA_RESERVA, HR_INICIO, HR_FIM

BOLETOS

- PK: ID_BOLETO
- FK: ID_MORADOR → MORADORES
- VL_BOLETO, DT_VENCIMENTO, STATUS

COMUNICADOS

- PK: ID_COMUNICADO
- TITULO, MENSAGEM, DT_COMUNICADO, HR_COMUNICADO, TIPO

CONTRATOS

- PK: ID_CONTRATO

- FK: ID_FORNECEDOR → FORNECEDORES
- DESCRICAO, DATA_INICIO, DATA_FIM, VALOR

CONTRATOS_RH

- PK: ID_CONTRATO_RH
- FK: ID_FUNCIONARIO → FUNCIONARIOS
- DESCRICAO, DATA_INICIO, DATA_FIM, SALARIO_ACORDADO

VISITAS

- PK: ID_VISITA
- FK: ID_VISITANTE → VISITANTES
- FK: ID_UNIDADE → UNIDADES
- FK: ID_MORADO_AUTORIZACAO → MORADORES
- PLACA_VEICULO
- DATA_ENTRADA, DATA_SAIDA

CONTAS_PAGAR

- PK: ID_CONTA_PAGAR
- FK: ID_FORNECEDOR → FORNECEDORES
- DESCRICAO, VALOR, DATA_VENCIMENTO, STATUS

CONTAS_RECEBER

- PK: ID_CONTA_RECEBER
- FK: ID_MORADOR → MORADORES
- DESCRICAO, VALOR, DATA_VENCIMENTO, STATUS

PAGAMENTOS

- PK: ID_PAGAMENTO
- FK: ID_CONTA_PAGAR → CONTAS_PAGAR
- DATA_PAGAMENTO, VALOR_PAGO, FORMA_PAGAMENTO

RECEBIMENTOS

- PK: ID_RECEBIMENTO
- FK: ID_CONTA_RECEBER → CONTAS_RECEBER
- DATA_RECEBIMENTO, VALOR_RECEBIDO, FORMA_RECEBIMENTO

CONTA_CORRENTE

- PK: ID_CONTA_CORRENTE
- BANCO, AGENCIA, NUM_CONTA, SALDO_ATUAL

MOV_CONTA_CORRENTE

- PK: ID_MOVIMENTO
- FK: ID_CONTA_CORRENTE → CONTA_CORRENTE
- ID_CONTA (referência dinâmica)
- ORIGEM_CONTA (identifica se ID_CONTA é de CONTAS_PAGAR ou CONTAS_RECEBER)
- TIPO_MOVIMENTO (Crédito/Débito), VALOR, DATA_MOVIMENTO, HR_MOVIMENTO, DESCRICAO

Relacionamentos e Cardinalidades do modelo:

- **PESSOAS 1:N CONTATOS**
- **PESSOAS 1:N ENDERECOS**
- **PESSOAS 1:1 MORADORES** (cada morador é uma pessoa)
- **PESSOAS 1:1 FUNCIONARIOS**
- **PESSOAS 1:1 FORNECEDORES**
- **PESSOAS 1:1 VISITANTES**
- **UNIDADES 1:N MORADORES**
- **UNIDADES 1:N VISITAS**
- **AREAS_COMUNS 1:N RESERVAS**
- **MORADORES 1:N RESERVAS**
- **MORADORES 1:N BOLETOS**
- **FORNECEDORES 1:N CONTRATOS**
- **FUNCIONARIOS 1:N CONTRATOS_RH**
- **VISITANTES 1:N VISITAS**
- **UNIDADES 1:N VISITAS**
- **MORADORES 1:N VISITAS** (autorização)
- **FORNECEDORES 1:N CONTAS_PAGAR**
- **MORADORES 1:N CONTAS_RECEBER**
- **CONTAS_PAGAR 1:N PAGAMENTOS**
- **CONTAS_RECEBER 1:N RECEBIMENTOS**
- **CONTA_CORRENTE 1:N MOV_CONTA_CORRENTE**

Tabelas Associativas Necessárias

- **MOV_CONTA_CORRENTE** já funciona como tabela associativa entre **CONTA_CORRENTE** e **CONTAS_PAGAR/CONTAS_RECEBER**.
- Não há necessidade de outras tabelas associativas explícitas, pois os relacionamentos já estão bem definidos via FKs.

O banco implementado permitirá que o sistema tenha:

Cadastros básicos: Pessoas, contatos, endereços, moradores, funcionários, fornecedores e visitantes.

Gestão operacional: Unidades, áreas comuns, reservas, comunicados e visitas.

Gestão financeira: Boletos, contas a pagar/receber, pagamentos, recebimentos, conta corrente e movimentações.

Gestão contratual: Contratos com fornecedores e contratos de RH.

4. Gerar o Modelo ER

No Workbench:

Após executar os scripts sql, utilize o recurso de engenharia reversa para desenhar o diagrama.

Entregável

Banco DBCONDOMINIO criado.

Exportado do Workbench

Diagrama ER pronto no Workbench.

Exportar o diagrama ER em PDF pelo Workbench.

Script SQL utilizado para criar o banco.

Em arquivo “txt”.

```

-- Criar Banco
CREATE DATABASE DBCONDOMINIO;
\c DBCONDOMINIO;

-- =====
-- TABELA PESSOAS
-- =====
CREATE TABLE PESSOAS (
  ID_PESSOA SERIAL PRIMARY KEY,
  NOME VARCHAR(150) NOT NULL,
  TIPO_PESSOA VARCHAR(20) NOT NULL, -- FISICA ou JURIDICA
  CPF_CNPJ VARCHAR(20) UNIQUE NOT NULL,
  DATA_CADASTRO DATE NOT NULL
);

-- =====
-- TABELAS DE APOIO (CONTATOS, ENDERECOS)
-- =====
CREATE TABLE CONTATOS (
  ID_CONTATO SERIAL PRIMARY KEY,
  ID_PESSOA INT REFERENCES PESSOAS(ID_PESSOA),
  TIPO_CONTATO VARCHAR(20) NOT NULL, -- TELEFONE, EMAIL, WHATSAPP
  VALOR_CONTATO VARCHAR(100) NOT NULL
);

CREATE TABLE ENDERECOS (
  ID_ENDERECO SERIAL PRIMARY KEY,
  ID_PESSOA INT REFERENCES PESSOAS(ID_PESSOA),
  LOGRADOURO VARCHAR(150),
  NUMERO VARCHAR(10),
  BAIRRO VARCHAR(100),
  CIDADE VARCHAR(100),
  UF VARCHAR(2),
  CEP VARCHAR(10)
);

-- =====
-- MORADORES, FUNCIONARIOS, FORNECEDORES, VISITANTES
-- =====
CREATE TABLE UNIDADES (
  ID_UNIDADE SERIAL PRIMARY KEY,
  NUM_UNIDADE VARCHAR(10) NOT NULL,
  BLOCO VARCHAR(10) NOT NULL,
  TIPO VARCHAR(20),
  AREA_TOTAL DECIMAL(10,2)
);

CREATE TABLE MORADORES (
  ID_MORADOR SERIAL PRIMARY KEY,
  ID_PESSOA INT REFERENCES PESSOAS(ID_PESSOA),
  ID_UNIDADE INT REFERENCES UNIDADES(ID_UNIDADE)
);

```

```
CREATE TABLE FUNCIONARIOS (  
  ID_FUNCIONARIO SERIAL PRIMARY KEY,  
  ID_PESSOA INT REFERENCES PESSOAS(ID_PESSOA),  
  FUNCAO VARCHAR(50),  
  DATA_ADMISSAO DATE,  
  SALARIO DECIMAL(10,2)  
);
```

```
CREATE TABLE FORNECEDORES (  
  ID_FORNECEDOR SERIAL PRIMARY KEY,  
  ID_PESSOA INT REFERENCES PESSOAS(ID_PESSOA),  
  AREA_ATUACAO VARCHAR(100)  
);
```

```
CREATE TABLE VISITANTES (  
  ID_VISITANTE SERIAL PRIMARY KEY,  
  ID_PESSOA INT REFERENCES PESSOAS(ID_PESSOA),  
  DOCUMENTO VARCHAR(20)  
);
```

```
-- =====  
-- ÁREAS COMUNS E RESERVAS  
-- =====
```

```
CREATE TABLE AREAS_COMUNS (  
  ID_AREA_COMUM SERIAL PRIMARY KEY,  
  NOME_AREA VARCHAR(50) NOT NULL,  
  DESCR_AREA TEXT,  
  CAPACIDADE INT  
);
```

```
CREATE TABLE RESERVAS (  
  ID_RESERVA SERIAL PRIMARY KEY,  
  ID_AREA_COMUM INT REFERENCES AREAS_COMUNS(ID_AREA_COMUM),  
  ID_MORADOR INT REFERENCES MORADORES(ID_MORADOR),  
  DATA_RESERVA DATE NOT NULL,  
  HR_INICIO TIME,  
  HR_FIM TIME  
);
```

```
-- =====  
-- BOLETOS E COMUNICADOS  
-- =====
```

```
CREATE TABLE BOLETOS (  
  ID_BOLETO SERIAL PRIMARY KEY,  
  ID_MORADOR INT REFERENCES MORADORES(ID_MORADOR),  
  VL_BOLETO DECIMAL(10,2) NOT NULL,  
  DT_VENCIMENTO DATE NOT NULL,  
  STATUS VARCHAR(20) -- PAGO, ABERTO, ATRASADO  
);
```

```
CREATE TABLE COMUNICADOS (  
  ID_COMUNICADO SERIAL PRIMARY KEY,  
  ID_MORADOR INT REFERENCES MORADORES(ID_MORADOR),  
  ID_FUNCIONARIO INT REFERENCES FUNCIONARIOS(ID_FUNCIONARIO),  
  DATA_COMUNICADO DATE,  
  CONTEUDO TEXT  
);
```

```

ID_COMUNICADO SERIAL PRIMARY KEY,
TITULO VARCHAR(100) NOT NULL,
MENSAGEM TEXT NOT NULL,
DT_COMUNICADO DATE NOT NULL,
HR_COMUNICADO TIME,
TIPO VARCHAR(30)
);

```

```

-- =====
-- CONTRATOS
-- =====

```

```

CREATE TABLE CONTRATOS (
  ID_CONTRATO SERIAL PRIMARY KEY,
  ID_FORNECEDOR INT REFERENCES FORNECEDORES(ID_FORNECEDOR),
  DESCRICAO TEXT,
  DATA_INICIO DATE,
  DATA_FIM DATE,
  VALOR DECIMAL(10,2)
);

```

```

CREATE TABLE CONTRATOS_RH (
  ID_CONTRATO_RH SERIAL PRIMARY KEY,
  ID_FUNCIONARIO INT REFERENCES FUNCIONARIOS(ID_FUNCIONARIO),
  DESCRICAO TEXT,
  DATA_INICIO DATE,
  DATA_FIM DATE,
  SALARIO_ACORDADO DECIMAL(10,2)
);

```

```

-- =====
-- VISITAS
-- =====

```

```

CREATE TABLE VISITAS (
  ID_VISITA SERIAL PRIMARY KEY,
  ID_VISITANTE INT REFERENCES VISITANTES(ID_VISITANTE),
  ID_UNIDADE INT REFERENCES UNIDADES(ID_UNIDADE),
  ID_MORADOR_AUTORIZACAO INT REFERENCES MORADORES(ID_MORADOR),
  PLACA_VEICULO VARCHAR(10),
  DATA_ENTRADA TIMESTAMP,
  DATA_SAIDA TIMESTAMP
);

```

```

-- =====
-- FINANCEIRO
-- =====

```

```

CREATE TABLE CONTAS_PAGAR (
  ID_CONTA_PAGAR SERIAL PRIMARY KEY,
  ID_FORNECEDOR INT REFERENCES FORNECEDORES(ID_FORNECEDOR),
  DESCRICAO TEXT,
  VALOR DECIMAL(10,2),
  DATA_VENCIMENTO DATE,
  STATUS VARCHAR(20)
);

```


);

```
CREATE TABLE CONTAS_RECEBER (  
  ID_CONTA_RECEBER SERIAL PRIMARY KEY,  
  ID_MORADOR INT REFERENCES MORADORES(ID_MORADOR),  
  DESCRICAO TEXT,  
  VALOR DECIMAL(10,2),  
  DATA_VENCIMENTO DATE,  
  STATUS VARCHAR(20)  
);
```

```
CREATE TABLE PAGAMENTOS (  
  ID_PAGAMENTO SERIAL PRIMARY KEY,  
  ID_CONTA_PAGAR INT REFERENCES CONTAS_PAGAR(ID_CONTA_PAGAR),  
  DATA_PAGAMENTO DATE,  
  VALOR_PAGO DECIMAL(10,2),  
  FORMA_PAGAMENTO VARCHAR(30)  
);
```

```
CREATE TABLE RECEBIMENTOS (  
  ID_RECEBIMENTO SERIAL PRIMARY KEY,  
  ID_CONTA_RECEBER INT REFERENCES CONTAS_RECEBER(ID_CONTA_RECEBER),  
  DATA_RECEBIMENTO DATE,  
  VALOR_RECEBIDO DECIMAL(10,2),  
  FORMA_RECEBIMENTO VARCHAR(30)  
);
```

```
CREATE TABLE CONTA_CORRENTE (  
  ID_CONTA_CORRENTE SERIAL PRIMARY KEY,  
  BANCO VARCHAR(50),  
  AGENCIA VARCHAR(20),  
  NUM_CONTA VARCHAR(20),  
  SALDO_ATUAL DECIMAL(12,2)  
);
```

```
CREATE TABLE MOV_CONTA_CORRENTE (  
  ID_MOVIMENTO SERIAL PRIMARY KEY,  
  ID_CONTA_CORRENTE INT REFERENCES  
CONTA_CORRENTE(ID_CONTA_CORRENTE),  
  ID_CONTA INT, -- pode ser ID_CONTA_PAGAR ou ID_CONTA_RECEBER  
  ORIGEM_CONTA VARCHAR(20), -- PAGAR ou RECEBER  
  TIPO_MOVIMENTO VARCHAR(20), -- CREDITO ou DEBITO  
  VALOR DECIMAL(10,2),  
  DATA_MOVIMENTO DATE,  
  HR_MOVIMENTO TIME,  
  DESCRICAO TEXT  
);
```